UNIVERSIDAD PRIVADA “FRANZ TAMAYO”

PROGRAMACION II

ESTUDIANTE:

Andreia Fernanda Jarjury Rojas

DOCENTE:

Ing. DIEGO PATRICK CARDENAS SEJAS

2026

I. Arquitectura y Programación Orientada a Objetos

1. Propósito y Funcionamiento General de NEXUS

NEXUS v1.1 es una aplicación interactiva desarrollada en C# orientada a objetos (POO). El sistema simula la exploración espacial, la gestión de un equipo de exploradores en un mapa orbital 2D y la ejecución de misiones de rescate. Permite registrar exploradores, monitorear su estado vital/energético y realizar búsquedas optimizadas mediante LINQ.

2. Estructura del Proyecto y Responsabilidades de sus Clases

- Program.cs: Punto de entrada de la aplicación. Contiene el bucle principal de la interfaz por consola y gestiona el menú del sistema.
- Explorador.cs: Representa la entidad principal del sistema. Modela los atributos del explorador (Nombre, Salud, Energía, Posición X/Y, Estado) y encapsula las acciones de movimiento y rescate.
- Form1.cs / NEXUS.UI: Componente de interfaz de usuario para la representación visual y mapas del sistema.

3. Diagrama de Clases (Texto/Estructura)

Plaintext

```
+-------------------------------------------------------+
|                      Explorador                       |
+-------------------------------------------------------+
| - nombre: string                                      |
| - salud: int                                          |
| - energia: int                                        |
| - posX: int                                           |
| - posY: int                                           |
+-------------------------------------------------------+
| + Explorador(nombre, salud, energia, posX, posY)      |
| + Mover(deltaX, deltaY): void                         |
| + MisionDeRescate(objetivo: Explorador): void         |
| + MostrarEstado(): void                               |
+-------------------------------------------------------+
```

4. Diferencias entre Programación Estructurada y POO

- Programación Estructurada: Se centra en funciones y procedimientos que operan sobre datos separados. El flujo de control es secuencial.
- Programación Orientada a Objetos (POO): Agrupa datos (atributos) y comportamientos (métodos) en unidades autónomas llamadas objetos, promoviendo la reutilización, modularidad y encapsulamiento.

II. Fundamentos de Clases y Objetos

1. Clases, Objetos e Instanciación

- Clase: Plantilla o molde conceptual (Explorador).
- Objeto: Instancia concreta en memoria (Explorador e1 = new Explorador("Alfa", 100, 80, 0, 0);).

2. Atributos, Campos, Propiedades y Métodos

- Campos/Atributos: Variables privadas dentro de la clase (private int salud;).
- Propiedades: Mecanismos de acceso seguro (public int Salud { get => salud; set => salud = value; }).
- Métodos: Operaciones que definen el comportamiento (public void Mover(...)).

3. Constructores y Sobrecarga

Permiten inicializar los objetos al ser creados.

```
C#

// Constructor principal
public Explorador(string nombre, int salud, int energia, int posX, int posY)
{
    this.nombre = nombre;
    this.salud = salud;
    this.energia = energia;
    this.posX = posX;
    this.posY = posY;
}

// Sobrecarga de constructor (complementario)
public Explorador(string nombre) : this(nombre, 100, 100, 0, 0) { }
```

4. Encapsulamiento y Modificadores de Acceso

El uso de private protege los datos de modificaciones indebidas desde fuera de la clase. El acceso controlado se realiza mediante propiedades públicas con validación.

III. Herencia, Polimorfismo e Interfaces

(Nota: Conceptos desarrollados mediante ejemplos complementarios aplicados a NEXUS)

1. Clases Base y Derivadas (Herencia)

```
public class ExploradorEspecializado : Explorador
{
    public string Especialidad { get; set; }
    public ExploradorEspecializado(string nombre, string espec) : base(nombre)
    {
        this.Especialidad = espec;
    }
}
```

2. Métodos Virtual y Override (Polimorfismo)

Permiten a las clases derivadas modificar el comportamiento de la clase base.

```
// En clase base:
public virtual void RealizarAccion() => Console.WriteLine("Accion estandar");

// En clase derivada:
public override void RealizarAccion() => Console.WriteLine("Accion especializada");
```

3. Interfaces

Definen contratos de comportamiento sin implementación.

```
public interface IRescatable
{
    void EjecutarRescate();
}
```

IV. Colecciones y LINQ

NEXUS utiliza List<Explorador> para gestionar la tripulación.

Consultas LINQ Utilizadas

```
// Filtrar exploradores con salud baja mediante Expresión Lambda
var criticos = listaExploradores.Where(e => e.Salud < 30).ToList();

// Ordenar exploradores por energía descendente
var ordenados = listaExploradores.OrderByDescending(e => e.Energia).ToList();
```

V. Análisis del Funcionamiento

1. Flujograma de la Misión de Rescate

1. Inicio
2. Seleccionar Explorador Rescatista y Objetivo
3. ¿Rescatista tiene suficiente Energía?

- *Sí:* Reducir energía del rescatista, incrementar salud del objetivo $\rightarrow$ 4. Misión Exitosa.
- *No:* Mostrar mensaje de energía insuficiente $\rightarrow$ 4. Fin.

2. Ejemplo de Seguimiento de Estado

- Estado Inicial: Rescatista (Energía: 50), Objetivo (Salud: 20)
- Ejecución: Rescatista.MisionDeRescate(Objetivo)
- Estado Final: Rescatista (Energía: 30), Objetivo (Salud: 50)

VI. Preparación para el Examen (Preguntas Teóricas)

- ¿Qué beneficio aporta el encapsulamiento en la clase Explorador?

Evita que código externo modifique directamente campos críticos como salud o energia con valores inválidos (como números negativos).

- ¿Cuál es la diferencia entre un constructor por defecto y uno parametrizado?

El constructor parametrizado exige los datos necesarios al instanciar el objeto para garantizar que nazca en un estado válido.

- ¿Qué ocurre si un método en la clase base no se declara como virtual?

No podrá ser sobrescrito (override) por clases derivadas para aplicar polimorfismo dinámico.

- ¿Por qué se prefieren las colecciones List<T> sobre los arreglos tradicionales T[]?

Porque List<T> es dinámica, permitiendo agregar y eliminar elementos sin necesidad de redimensionar manualmente el tamaño del contenedor.

- ¿Qué devuelve la consulta lista.Where(e => e.Salud > 50)?

Devuelve un ejecutable IEnumerable<Explorador> filtrado con los objetos que cumplen la condición booleana especificada.

- ¿Qué sucedería si se elimina la palabra clave this en un constructor con parámetros del mismo nombre que los atributos?

Ocurriría una ambigüedad donde la variable se asignaría a sí misma, dejando los atributos de la clase sin inicializar (con valores por defecto).

VII. DIAGNÓSTICO PERSONAL DE APRENDIZAJE — PROMPT ESPEJO

Plaintext

PROMPT ESPEJO

Actúa como un evaluador de mi proceso de aprendizaje durante la misión NEXUS: DE CÓDIGO A CONOCIMIENTO.

Analiza nuestra conversación completa, prestando especial atención a mis aportes personales: preguntas, razonamientos, explicaciones, decisiones, errores y correcciones.

Genera un diagnóstico con estos apartados:

1. Fortalezas demostradas:

- Comprensión clara del flujo de trabajo de Git/GitHub Desktop y resolución de credenciales.
- Capacidad para estructurar refactorizaciones de código desde enfoques estructurados hacia POO.
- Aplicación práctica de LINQ y colecciones genéricas en C#.

2. Falencias observadas:

- Dependencia inicial en la resolución sintáctica de errores de compilación (como el error CS1513).
- Manejo de rutas locales del sistema operativo al configurar repositorios de Git.

3. Diagnóstico del Hito 3:

- POO, Clases y Objetos: Comprensión demostrada.
- Encapsulamiento y Constructores: Comprensión demostrada.
- LINQ y Colecciones: Comprensión demostrada.
- Herencia y Polimorfismo: Comprensión parcial (desarrollado conceptualmente).

4. Mi forma de aprender:

- Aprendizaje guiado, iterativo y práctico mediante resolución inmediata de bloqueos técnicos.

5. Plan de estudio:

- Prioridad 1: Practicar sintaxis de Interfaces y Clases Abstractas.
- Prioridad 2: Resolver ejercicios de trazado manual de flujo para exámen escrito.
- Prioridad 3: Repasar operadores LINQ de agregación (Sum, Average, Min, Max).

6. Conclusión:

Demostrada la capacidad de implementar un proyecto funcional POO, documentar adecuadamente la solución y gestionar el control de versiones en GitHub.